

Preparcial (20%) - Plataforma E-Commerce con Next.js y React Context

Objetivo general

Integrar conceptos fundamentales y de arquitectura en **React** y **Next.js (App Router)** para diseñar una aplicación web modular que gestione un catálogo de productos, permita la navegación fluida entre vistas estáticas y dinámicas, y mantenga un estado global compartido y persistente a lo largo de toda la experiencia de usuario, haciendo uso de:

- Enrutamiento de vistas y layouts persistentes con **App Router**.
- Manejo de estado local, efectos e interactividad en componentes de cliente.
- Comunicación de componentes y flujo unidireccional de datos mediante **props y callbacks**.
- Gestión y consumo de estado compartido utilizando **React Context API**.
- Consulta asíncrona a servicios web externos mediante peticiones HTTP.
- Tipado estricto de modelos y componentes con **TypeScript**.

Preparación del entorno

Para el desarrollo de este preparcial no se requiere desplegar servicios de backend locales. La aplicación consumirá directamente la API pública de **DummyJSON**.

1. Creación del proyecto base

Genere el proyecto utilizando Next.js, App Router y TypeScript:

```
npx create-next-app@latest shophub --typescript --eslint --app --src-dir --import-alias "@/*"
```

2. Endpoints por consumir

- **Listado del catálogo:** GET <https://dummyjson.com/products?limit=8&select=id,title,price,category,thumb>
- **Detalle de producto individual:** GET <https://dummyjson.com/products/{id}>

Enunciado del reto

Desarrollar la aplicación web **ShopHub**, una plataforma de comercio electrónico construida sobre la arquitectura App Router de Next.js.

La aplicación debe permitir a los usuarios consultar un catálogo de artículos obtenidos de forma asíncrona, explorar la información detallada de cualquier producto mediante navegación dinámica y administrar un carrito de compras global cuyos datos se mantengan sincronizados y persistentes a lo largo de todas las vistas sin recargas completas del navegador (SPA).

Especificación de pantallas y funcionalidades

1. Barra superior persistente (Header)

Componente de navegación visible en todas las rutas de la aplicación:

- **Identidad de la tienda:** título o marca con enlace de retorno hacia la vista principal.
- **Indicador del carrito:** contador dinámico que muestra en todo momento la cantidad acumulada de productos seleccionados, reflejando de inmediato cualquier adición realizada desde las vistas.

2. Vista del catálogo principal

Página de inicio encargada de presentar los artículos disponibles:

- **Carga y despliegue del catálogo:** consulta de los productos al servicio externo y presentación estructurada en tarjetas modulares.
- **Contenido por artículo:**
 - Miniatura del producto, nombre, categoría, precio y unidades en stock.
 - Enlace interactivo para acceder a la ficha de detalle individual del producto.
 - Botón de acción rápida para añadir el artículo directamente al carrito de compras global.

3. Vista de detalle de producto

Ruta dinámica parametrizada para consultar la información completa de un artículo específico a partir de su identificador:

- **Carga de información puntual:** petición asíncrona para obtener los datos extendidos del producto seleccionado.
- **Panel de información:**
 - Fotografía principal, nombre del producto, categoría o marca, precio y disponibilidad de stock.
 - Descripción detallada del producto.
 - Botón de acción para añadir el artículo al carrito desde esta vista.
 - Enlace o botón de retorno hacia el catálogo principal.

Requisitos de arquitectura y buenas prácticas

- **Estado global compartido:** implementar un contexto de React que centralice la información de los productos en el carrito y exponga los mecanismos necesarios para agregar nuevos ítems desde cualquier nivel de la interfaz.
- **Persistencia de navegación:** el estado global no debe reiniciarse al navegar entre el catálogo y las vistas de detalle.
- **Inmutabilidad:** garantizar la actualización adecuada de arreglos y objetos en memoria, evitando mutaciones directas y asegurando que React detecte los cambios de estado.
- **Modelado y tipado:** definir contratos claros con TypeScript para los datos del producto y las propiedades que intercambian los componentes.
- **Fronteras cliente/servidor:** identificar y marcar como componentes de cliente únicamente aquellos módulos que requieran interactividad, captura de eventos o hooks de estado/ciclo de vida.
- **Diseño de interfaz:** evitar aplicaciones sin diseño o poco usables. Se recomienda dejar la aplicación lo más organizada y visualmente adecuada posible.

Entrega

La entrega de este preparcial se realizará por medio de Bloque Neón, enviando el enlace al **repositorio público de GitHub** con la solución completa y funcional antes de la fecha límite establecida.

Nota para el examen parcial: este repositorio servirá como base de código obligatoria sobre la cual cada estudiante trabajará de forma individual durante el examen en vivo, donde se evaluará la extensión de la arquitectura, la creación de nuevas vistas y la manipulación avanzada del estado.

1. Enlace al Release de GitHub

- El código debe alojarse en un **repositorio público de GitHub**.
- Se debe generar un **GitHub Release** (etiqueta sugerida: `v1.0.0` o `preparcial-v1.0`) que congele el estado del proyecto antes de la fecha y hora límite de entrega.
- En Bloque Neón debe enviarse el enlace directo a dicho *Release*.

2. Video demostrativo

Adjuntar un enlace público a un video (YouTube en modo oculto o público, Loom, Google Drive con permisos abiertos, etc.) con una duración máxima de **4 a 7 minutos**, donde se demuestre con fluidez y claridad el funcionamiento de la aplicación:

- **Evidencia del tag o Release:** mostrar brevemente en el navegador la pestaña de GitHub en la sección **Releases**, señalando la versión creada (**v1.0.0**) y el commit hash asociado.
- **Evidencia del entorno local:** mostrar la terminal ejecutando **git status** y **git log -1** para evidenciar que el proyecto que se corre localmente (**npm run dev**) se encuentra en ese mismo commit y sin cambios pendientes de confirmar (*working tree clean*).
- **Flujo de navegación SPA:** mostrar la transición entre el catálogo principal (/) y la vista de detalle de al menos dos productos distintos (/productos/[id]), evidenciando que no existen recargas completas del navegador.
- **Sincronización del estado global:** agregar productos al carrito tanto desde la vista principal como desde la vista de detalle.
- **Actualización del Header:** señalar claramente cómo el contador del carrito en la barra superior se actualiza reactivamente en tiempo real con cada adición y cómo mantiene su valor intacto al cambiar entre rutas.
- **Manejo de errores y calidad:** mostrar que la consola del navegador se encuentra limpia de advertencias o errores críticos durante toda la interacción.

Nota para el examen parcial: el código congelado en este *Release* será la base de trabajo individual obligatoria sobre la cual cada estudiante trabajará en tiempo real durante el **Examen Parcial en vivo**.